

Attention Is All You Need

The Transformer — sequence transduction from attention alone, with recurrence and convolution stripped out

Authors • Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin (Google Brain / Google Research / Univ. of Toronto)

Venue • 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA • arXiv:1706.03762

Report language • English

Contents • 5 equations analyzed • 3 original figure crops • 3 re-typeset tables • 11 web-verified background items • 4 machine-verified code snippets

One-line summary. The paper removes recurrence (RNNs) and convolution (CNNs) entirely and builds an encoder—decoder from **self-attention alone** — the **Transformer**. Because every pair of positions is connected by a constant number of operations, training parallelizes massively, and the model set a new state of the art on WMT 2014 English—German and English—French translation at a fraction of the training cost.

How to read this report — Cornwell's 7 steps	2
Phase 0 — Background (ladder • related-work map)	3
Phase 1 — The Aerial View (Steps 1 • 2 • 3)	6
Phase 2 — The Interrogation (Steps 4 • 5)	10
Equation-by-equation analysis (5 cards)	11
Independent conclusions (Tables 2 • 3)	16
Common sticking points, Top 4	19
Phase 3 — The Verdict (Steps 6 • 7)	21
Final insight & 7-question self-check	22
Appendix — Run it yourself (4 verified snippets)	23
Glossary	23
References (with verification status)	25

How to read this report — Cornwell's 7-step method

Reading a paper well does not start with the equations on page one. The method Jacques Cornwell laid out in a *Nature* career column (2026) proceeds in three phases — **context** → **interrogation** → **verdict** — across seven steps. First grasp the big picture (Steps 1–3), then take the methodology and the math apart (Steps 4–5), and only at the end weigh the authors' claims against your own independent reading (Steps 6–7). Each section of this report follows one of those steps, and every step does two things: **(a)** it first explains what a reader is supposed to do at that step, and **(b)** it then shows the result of actually doing it for this paper. An experienced researcher spends one to two hours on a paper this way; this document performs those hours for you and shows the work.

Phase	Step	Guiding question
Phase 1 The Aerial View	Step 1. Gain a broad overview	Can the abstract and the key figure alone tell you the design, tools, and scope?
	Step 2. Find the core research question	What specific hypothesis is this paper testing?
	Step 3. Map the knowledge gap	What limits prior approaches, and where does this paper slot in?
Phase 2 The Interrogation	Step 4. Assess the methodology	Does the experimental design actually answer the question? (math • structure • controls)
	Step 5. Reach independent conclusions	Cover up the authors' interpretation — what do the results alone say?
Phase 3 The Verdict	Step 6. Reconcile conclusions	Do the authors' claims outrun their data?
	Step 7. Consider alternatives	Could confounders, other factors, or conflicts of interest explain the results?

PHASE 0 • PRELIMINARIES

Background — groundwork before the 7 steps

This section is not one of Cornwell's steps, but it is the foundation you need before you can judge the paper's math yourself. The **prerequisite ladder** below starts from the most primitive concept (the dot product) and stacks upward, in dependency order, all the way to the paper's own notation (e.g. $\text{softmax}(QK^T/\sqrt{d_k})V$). Each rung gives **the concept** → **a short explanation** → **a worked example with real numbers** → **which symbol, equation, or claim of the paper it unlocks**. External knowledge appears only if it passed the verification protocol in the [references](#).

Prerequisite ladder**1. Dot product as similarity**

The dot product $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ of two vectors grows when they point the same way — a **similarity score**.

Numbers: $[1, 2] \cdot [3, 4] = 1 \cdot 3 + 2 \cdot 4 = 11$, while $[1, 2] \cdot [-2, 1] = 0$ (orthogonal).

→ QK^T in Eq. (1): the query—key dot product measures "how much should this position look at that one".

2. Matrix multiplication and shape bookkeeping

An $(m \times k)$ matrix times a $(k \times n)$ matrix gives $(m \times n)$. Stacking queries into a matrix processes them all at once.

Numbers: 3 queries and 4 keys packed as $Q \in \mathbb{R}^{3 \times 64}$, $K \in \mathbb{R}^{4 \times 64}$ give $QK^T \in \mathbb{R}^{3 \times 4}$ — all 3×4 scores in one product.

→ why Eq. (1) uses packed matrices Q, K, V instead of single vectors (the heart of parallelization).

3. Soft max — scores to probabilities

Mapping a real vector $\mathbf{z}$ through $\text{softmax}(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$ yields positive **weights that sum to 1**.

Numbers: $\mathbf{z} = [2, 1, 0] \rightarrow e^{\mathbf{z}} \approx [7.39, 2.72, 1]$, total 11.11 → $[0.665, 0.245, 0.090]$ (sums to 1.000; machine-checked in the appendix).

→ $\text{softmax}(\cdot)$ in Eq. (1): normalizes dot-product scores into "how much of each value to take".

4. Weighted average (convex combination)

With weights $w_i \geq 0$, $\sum w_i = 1$, the sum $\sum_i w_i v_i$ is a **blend** of the values.

Numbers: with the weights above, $0.665 v_1 + 0.245 v_2 + 0.090 v_3$ — a blend dominated by v_1 .

→ "the output is a weighted sum of values" — the definition of attention (the trailing $\dots V$ in Eq. (1)).

5. Word embeddings

A learned look-up table maps each token to a d_{model} -dimensional real vector; similar words get nearby vectors.

Numbers: this paper uses $d_{\text{model}} = 512$, so each token is a 512-dim vector; a 37k vocabulary makes the table 37000×512 .

→ the Input/Output Embedding boxes in Figure 1, and §3.4's multiplication of embeddings by $\sqrt{d_{\text{model}}}$.

6. Encoder—decoder and autoregressive generation

The encoder maps the input $(x_1, \dots, x_n)$ to a continuous representation z ; the decoder emits the next token while looking at z and **only the tokens generated so far**.

Numbers: having generated "I am", the next step's input is "I am" and the target is "fine" — future tokens are invisible.

→ why the decoder has masked self-attention and why Figure 1 says "outputs (shifted right)".

7. Where attention came from (Bahdanau attention)

In RNN translation, instead of squeezing the whole input into one vector, attention let each output word **softly select** the relevant input positions. ^[K1]

The key difference: this paper pulls that idea out of the RNN — representations are built from attention alone, no recurrence.

→ the conceptual root of self-attention; the comparison target in §2 Background.

8. Residual connections and LayerNorm

Adding the input back to the output, $x + \text{Sublayer}(x)$ (residual ^[K11]), makes deep networks trainable; LayerNorm ^[K5] normalizes each sample's features to stabilize training.

Numbers: features $[2, 4, 6] \rightarrow \text{mean } 4, \text{std} \approx 1.63 \rightarrow \text{normalized } [-1.22, 0, 1.22]$.

→ every "Add & Norm" box in Figure 1, i.e. $\text{LayerNorm}(x + \text{Sublayer}(x))$.

9. Sine/cosine frequencies

$\sin(\text{pos}/T)$ oscillates with period $2\pi T$. Stacking sinusoids of different wavelengths gives every position a unique pattern.

Numbers: wavelengths grow geometrically from 2π to $10000 \cdot 2\pi$, building a 512-dim "positional fingerprint".

→ the positional-encoding formula $PE_{(\text{pos}, 2i)} = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$.

10. Variance and scaling — why divide by $\sqrt{d_k}$

The dot product of two independent d_k -dim vectors with zero-mean, unit-variance entries has mean 0 and variance d_k . Large variance saturates the softmax and kills gradients.

Numbers: with $d_k = 64$ the dot product's std is ≈ 8 ; dividing by $\sqrt{64} = 8$ restores variance to 1.

→ the scale factor $1/\sqrt{d_k}$ in Eq. (1) and the footnote that justifies it.

Related-work map — where this paper sits

Section §2 (Background) organizes the drive to "reduce sequential computation" into three lineages. Each lineage's **limitation** is precisely a design motivation for the Transformer.

Lineage	Representative work	Remaining limitation
① RNN seq2seq + attention	Sutskever et al. 2014 (seq2seq) ^[k2] , Bahdanau et al. 2014 (additive attention) ^[k1] , built on LSTMs ^[k3]	The hidden state h_t depends on $h_{t-1} \rightarrow O(n)$ sequential operations along the sequence; training cannot parallelize within an example.
② CNN seq2seq	ByteNet ^[k6] , ConvS2S ^[k4] , Extended Neural GPU	Parallel across positions, but the path connecting two positions grows with their distance (linearly for ConvS2S, logarithmically for ByteNet) $\rightarrow$ long-range dependencies are hard to learn.
③ Self-attention inside RNNs / memory networks	structured attention, end-to-end memory networks	Uses self-attention but still coupled to recurrence $\rightarrow$ sequentiality remains.
$\rightarrow$ This paper (Transformer)	A pure self-attention encoder—decoder	Any two positions connected in a constant number of operations ($O(1)$) . Fully parallel, no recurrence or convolution. Simplifies the architecture itself, unlike large-scale conditional computation (MoE ^[k7]).

PHASE 1 • THE AERIAL VIEW

Context — Steps 1 • 2 • 3

STEP 1

Gain a broad overview

WHAT TO DO AT THIS STEP

Cornwell: "Resist the urge to dive straight into dense methodology. Instead, scan the abstract and review the figures." Dissect the abstract sentence by sentence into **background / method / result / significance**, and read the key figure (the architecture) element by element to fix the design, tools, and scope first.

The abstract, sentence by sentence

Role	What the abstract asserts (gist)
Background	Dominant sequence-transduction models are complex RNNs or CNNs with an encoder and decoder; the best also connect the two through attention.
Method	Propose the Transformer — a simple architecture based on attention only , dispensing with recurrence and convolution entirely.
Result	WMT 2014 EN—DE 28.4 BLEU (over 2 BLEU above the previous best incl. ensembles); single-model EN—FR 41.8 BLEU after 3.5 days on 8 GPUs — a fraction of the prior training cost.
Significance	More parallelizable, faster to train, and generalizes well to English constituency parsing.

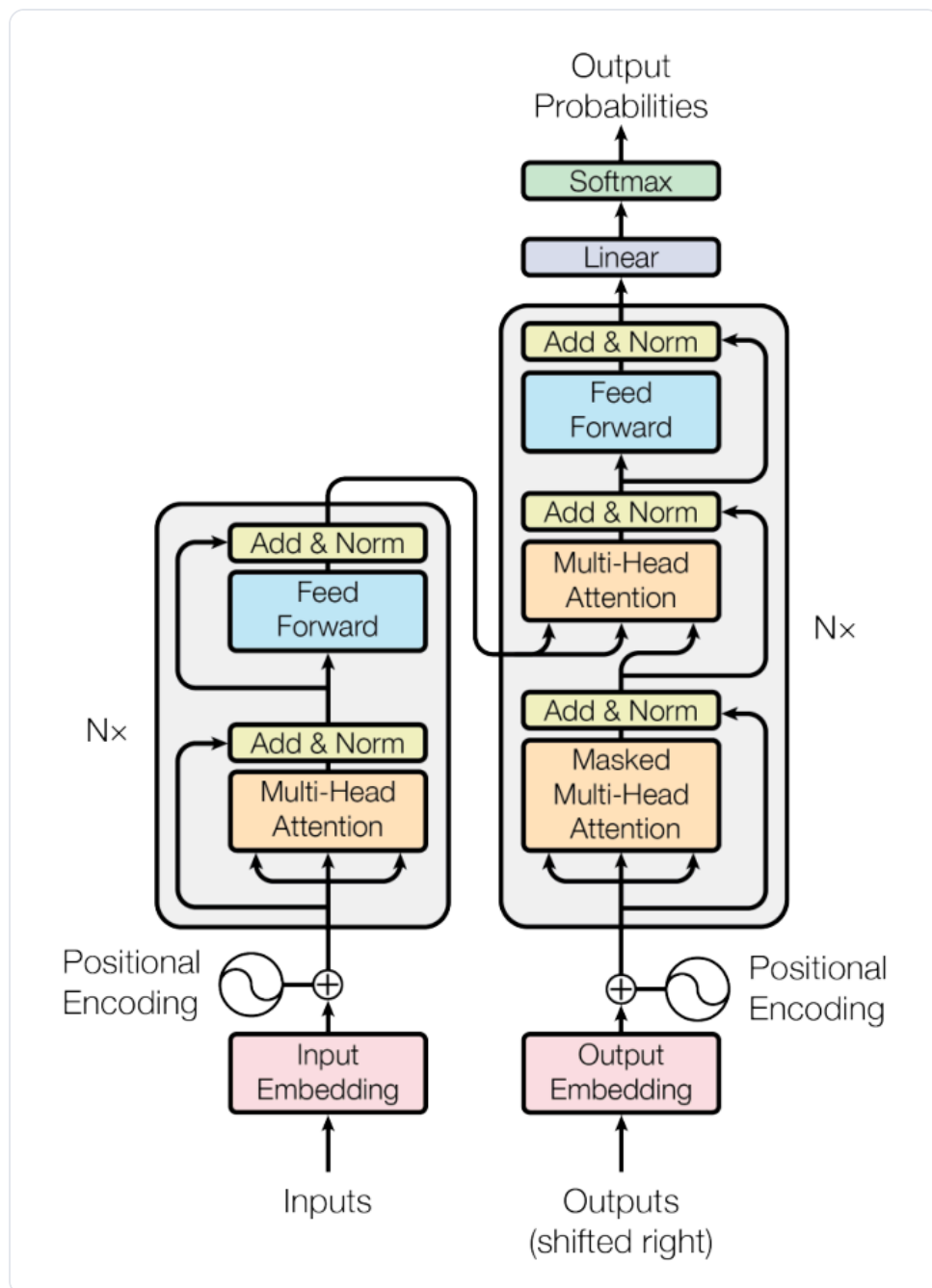


Figure 1 — The full Transformer. Encoder stack on the left, decoder stack on the right; both repeat $N = 6$ times.

What this figure draws

The complete data flow: an encoder (left) turning the input sentence into representations, and a decoder (right) producing next-token probabilities from those representations plus its own output so far.

Reading order

① Bottom left: **Inputs** → Input Embedding maps tokens to 512-dim vectors. ② **Positional Encoding** is added ($\oplus$) to inject order. ③ **Multi-Head Attention** (encoder self-attention) → **Add & Norm** → **Feed Forward** → **Add & Norm**; this layer repeats N times. ④ The decoder starts from **Masked Multi-Head Attention** (future blocked), then **encoder—decoder attention** taking the encoder output as keys/values, then **Feed Forward**. ⑤ At the top, **Linear + Softmax** gives next-word probabilities (Output Probabilities).

In-figure symbols

$N \times$: the same layer stacked $N = 6$ times. $\oplus$: element-wise addition of embedding and positional encoding.

"Outputs (shifted right)": the target sequence fed in shifted one step for autoregressive training. The arrows that split and rejoin at Add & Norm are the residual connections.

What to observe

Nowhere in the encoder or decoder is there a recurrent arrow (a loop back through time) or a convolution — only stacked attention + FFN blocks. That is "attention is all you need", in picture form.

Connections

Multi-Head Attention blocks = Eq. (1) + the MultiHead formula; Feed Forward = Eq. (2); the $\oplus$ at the bottom = the positional-encoding formulas; Add & Norm = **LayerNorm**($x + \text{Sublayer}(x)$).

STEP 2**Find the core research question****WHAT TO DO AT THIS STEP**

Locate the specific hypothesis — usually at the end of the introduction — and present it as **original quote** + **plain-language unpacking**; keep returning to it throughout. If it is never stated, reconstruct it and say so.

The final paragraph of the introduction (§1) effectively declares the paper's thesis:

"In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality ..."

Unpacked — the hypothesis under test. *Strip out recurrence and convolution entirely and build a sequence-transduction model from self-attention alone; then (a) translation quality does not degrade — it surpasses the state of the art — and (b) with sequential dependencies gone, training is much faster.* The rest of this report tracks whether claims (a) and (b) are actually supported by the math, the architecture, and the experiments.

STEP 3

Map the knowledge gap

WHAT TO DO AT THIS STEP

From the introduction and related work, summarize the state of the field, the lineages of prior approaches and their limits, and the puzzle piece this paper adds. (The detailed lineage table is in [Phase 0's related-work map](#).)

The gap is crisp. RNNs cannot parallelize within an example because of their **sequentiality** — factorizing along positions means h_t waits for h_{t-1} — and memory constraints limit batching across long sequences. CNNs parallelize, but their **path length grows with distance** (linearly for ConvS2S, logarithmically for ByteNet), making long-range dependencies harder to learn. Attention had already proven itself for modeling dependencies regardless of distance, yet — as the paper itself notes, "in all but a few cases" — it was always used *together with* a recurrent network, so the sequential bottleneck remained.

The open slot is therefore precise: **"can attention alone, with no recurrence, build the representations?"** The Transformer is that piece — any two positions linked in $O(1)$ operations, fully parallel, with §4's complexity analysis (Table 1 below) quantifying the advantage. The paper also probes how far the piece generalizes beyond translation, testing the same architecture on English constituency parsing (§6.3) with competitive results — a first hint that the design is not translation-specific.

PHASE 2 • THE INTERROGATION

Detailed evaluation — Steps 4 • 5

STEP 4

Methodology assessment (1) — method dissection

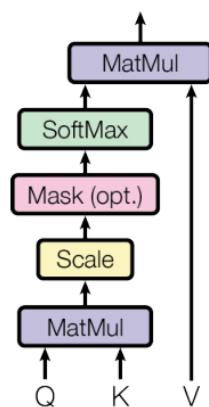
WHAT TO DO AT THIS STEP

Check that the experimental design answers the research question: appropriateness of tools, models, and data; sample size and repeats (for ML: seeds and repeat runs); positive/negative controls (for ML: baselines and ablations); data and code availability. This step has two large parts: **method dissection** and **equation-by-equation analysis**.

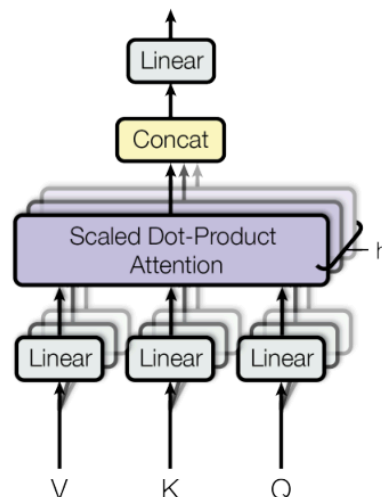
The Transformer in pipeline order

(1) Embedding + positional encoding. Tokens are embedded into $d_{\text{model}} = 512$ vectors (multiplied by $\sqrt{d_{\text{model}}}$) and a positional encoding is added — without recurrence, this is the only way the model knows word order. **(2) Encoder layers $\times 6$.** Each layer = [multi-head self-attention] + [position-wise FFN], each sublayer wrapped in residual + LayerNorm (Add & Norm). **(3) Decoder layers $\times 6$.** Each layer = [masked self-attention (future positions set to $-\infty$ to preserve autoregression)] + [encoder—decoder attention (queries from the decoder, keys/values from the encoder output)] + [FFN]. **(4) Output.** A final Linear produces vocabulary logits, softmax turns them into probabilities (weights shared with the embedding matrix).

Scaled Dot-Product Attention



Multi-Head Attention

Figure 2 — (left) The operations inside Scaled Dot-Product Attention; (right) Multi-Head Attention running h of them in parallel.

What this figure draws

Left: the five-operation stack by which one attention unit maps Q, K, V to an output. Right: the same unit run h times in parallel through different learned projections, then merged.

Reading order

Left, bottom → top: ① MatMul (QK^T , similarity scores) → ② Scale ($\div \sqrt{d_k}$) → ③ Mask (opt.) (only in the decoder: future positions to $-\infty$) → ④ SoftMax (weights) → ⑤ MatMul (weights $\times V$, the weighted sum). **Right, bottom → top:** V, K, Q each pass through Linear layers h times → h Scaled Dot-Product Attentions run in parallel → Concat → a final Linear (W^O) back to d_{model} .

In-figure symbols

Q (queries), K (keys), V (values): matrices obtained by projecting the input. h : the number of parallel heads (8 in this paper). "Mask (opt.)": optional masking, switched on only in decoder self-attention.

What to observe

Why split into h small heads instead of one big one? A single attention averages and smears information.

Shrinking to $d_k = d_v = d_{\text{model}}/h = 64$ and running h of them keeps total compute near single-head cost while attending to several **representation subspaces** at once.

Connections

The whole left panel = Eq. (1). The whole right panel = the MultiHead formula (card below). The Mask block = §3.2.3's preservation of decoder autoregression.

Methodology assessment (2) — equation-by-equation analysis

The paper has three numbered equations; two unnumbered ones (MultiHead and the positional encoding) are decisive for understanding, so all five get a card.

Scaled Dot-Product Attention — Eq. (1)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

SYMBOLS

Symbol	Type / shape	Meaning
Q	matrix $\mathbb{R}^{n \times d_k}$	packed queries — "what each position is looking for"
K	matrix $\mathbb{R}^{m \times d_k}$	packed keys — "the label each position advertises"
V	matrix $\mathbb{R}^{m \times d_v}$	packed values — "the content actually retrieved"
QK^T	matrix $\mathbb{R}^{n \times m}$	dot-product (similarity) scores for every query—key pair
d_k	scalar	key/query dimension (64 in this paper)
$\sqrt{d_k}$	scalar	variance-normalizing scale factor

TERM-BY-TERM ROLES

$QK^\top$	Measures how aligned each query is with each key; a large value says "look there". Remove it and the model has no basis for deciding where to attend.
$/\sqrt{d_k}$	As d_k grows, dot-product variance inflates like d_k , saturating the softmax (vanishing gradients). Dividing by $\sqrt{d_k}$ restores variance to ≈ 1 ; without it, training stalls at large d_k .
$\text{softmax}(\cdot)$	Turns scores into weights summing to 1 — per row, "how much of each value to take".
$\dots V$	Blends the values with those weights. The output is a mixture of the content at relevant positions.

INTUITION

"Hold your question (Q) against the labels (K) on a row of drawers, then pull out each drawer's contents (V) in proportion to the match." With numbers: if a query scores $[8, 0]$ against two keys (scaled: $[1, 0]$), softmax gives $[0.73, 0.27]$ and the output is $0.73 v_1 + 0.27 v_2$.

DEPENDENCIES

Prerequisite ladder (dot product • softmax • weighted average) → **this equation** → substituted per-head into the MultiHead formula; in the decoder, the Mask slips in before the softmax.

Multi-Head Attention — (unnumbered, §3.2.2)

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where } \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

SYMBOLS

Symbol	Type / shape	Meaning
h	scalar	number of heads (8 in this paper)
W_i^Q	matrix $\mathbb{R}^{d_{\text{model}} \times d_k}$	query projection of head i
W_i^K	matrix $\mathbb{R}^{d_{\text{model}} \times d_k}$	key projection of head i
W_i^V	matrix $\mathbb{R}^{d_{\text{model}} \times d_v}$	value projection of head i
W^O	matrix $\mathbb{R}^{hd_v \times d_{\text{model}}}$	output projection returning the concat to d_{model}
head_i	matrix $\mathbb{R}^{n \times d_v}$	attention output in the i -th subspace

TERM-BY-TERM ROLES

QW_i^Q, KW_i^K, VW_i^V	Project the same input from a different learned angle per head, so different heads can watch different relations (syntax, coreference, ...). Without the projections, all heads would see the same thing.
$\text{Concat}(\dots)$	Joins the h head outputs ($d_v = 64$ each) into one $hd_v = 512$ vector, pooling the subspace views.
W^O	Mixes the pooled information back to d_{model} , matching shapes for the residual addition.

INTUITION

Instead of one reader reading a sentence from a single perspective, eight readers read it simultaneously from different perspectives (subject—verb agreement, pronoun reference, adjacency, ...) and pool their summaries. Because $d_k = d_v = 512/8 = 64$, all eight together cost about as much as one full-width attention.

DEPENDENCIES

Eq. (1) is the inside of every **head_i** → **this formula** → used by all three attentions in Figure 1 (encoder self / decoder masked self / encoder—decoder).

Position-wise Feed-Forward Network — Eq. (2)

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

SYMBOLS

Symbol	Type / shape	Meaning
x	vector $\mathbb{R}^{d_{\text{model}}}$	one position's representation (512-dim)
W_1, b_1	$\mathbb{R}^{512 \times 2048}, \mathbb{R}^{2048}$	expansion layer (inner dimension $d_{ff} = 2048$)
W_2, b_2	$\mathbb{R}^{2048 \times 512}, \mathbb{R}^{512}$	contraction layer (back to 512)
$\max(0, \cdot)$	function (ReLU)	nonlinearity clipping negatives to zero

TERM-BY-TERM ROLES

$xW_1 + b_1$	Widens 512 → 2048, lifting each position into a richer feature space.
$\max(0, \cdot)$	Injects nonlinearity — without it the two linear layers collapse into one and the expressive power vanishes.
$\dots W_2 + b_2$	Contracts 2048 → 512 to match shapes for the residual addition and the next layer.

INTUITION

If attention mixes information *between* positions, the FFN reprocesses each position **independently and identically** (the same W_1, W_2 at every position — two convolutions with kernel size 1). Zero cross-position interaction is exactly its division of labor with attention.

DEPENDENCIES

MultiHead output (after Add & Norm) → **this equation** → another Add & Norm → next layer. One per layer in both encoder and decoder.

Positional Encoding — (unnumbered, §3.5)

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

SYMBOLS

Symbol	Type	Meaning
pos	integer scalar	position in the sequence (0, 1, 2, ...)
i	integer scalar	dimension index (even → sin, odd → cos)
d_{model}	scalar	embedding width (512) — PE shares it so the two can be added
$10000^{2i/d_{\text{model}}}$	scalar	per-dimension wavelength (period) control

TERM-BY-TERM ROLES

$pos/10000^{2i/d_{\text{model}}}$	Low i → short wavelength (fast oscillation), high i → long wavelength. Wavelengths spread geometrically from 2π to $10000 \cdot 2\pi$, giving each position a unique pattern.
sin / cos pairs	With each frequency present as a sin—cos pair, PE_{pos+k} is a linear transform of PE_{pos} for any fixed offset k — making relative positions easy to learn.

INTUITION

Encode "which position am I" as the combined angles of clock hands spinning at many speeds. Learned positional embeddings performed nearly identically (Table 3 (E)); the sinusoids were chosen hoping they extrapolate to sequences **longer than those seen in training**.

DEPENDENCIES

Input embedding ($\times \sqrt{d_{\text{model}}}$) → **element-wise addition with this** → the first layer of both stacks. The only channel for order information in a recurrence-free model.

Learning-rate schedule — Eq. (3)

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

SYMBOLS

Symbol	Type	Meaning
<i>step_num</i>	integer	training steps so far
<i>warmup_steps</i>	integer (=4000)	length of the linear warm-up window
<i>d_{model}</i>	scalar (=512)	model width — sets the overall learning-rate scale

TERM-BY-TERM ROLES

<i>step_num · warmup_steps^{-1.5}</i>	Dominates during warm-up (<i>step</i> < <i>warmup</i>) → learning rate grows linearly , avoiding early instability from large steps.
<i>step_num^{-0.5}</i>	Dominates afterwards → learning rate decays as the inverse square root , fine-tuning late in training.
<i>min(·, ·)</i>	Switches between the two regimes automatically; they cross exactly at <i>step</i> = <i>warmup_steps</i> .
<i>d_{model}^{-0.5}</i>	Larger models get smaller learning rates, matching the overall scale.

INTUITION

"Warm slowly, then cool gently." The rate peaks at *warmup* = 4000 and decays smoothly after; used with Adam^[k8] ($\beta_1=0.9, \beta_2=0.98, \epsilon=10^{-9}$).

DEPENDENCIES

An **optimization recipe** independent of the architecture. Defined in §5.3; every training result in Tables 2—3 depends on it.

Complexity evidence — what Table 1 says

Section §4 quantifies the case for self-attention along three axes. Notation: *n* = sequence length, *d* = representation width, *k* = convolution kernel, *r* = restricted-attention neighborhood.

Layer type	Complexity per layer	Sequential ops	Max path length
------------	----------------------	----------------	-----------------

Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Re-typeset from the paper's Table 1; values verified cell by cell against the original.

The point: self-attention needs $O(1)$ sequential operations and has $O(1)$ path length — any two positions connect in one hop (RNNs are $O(n)$ on both). Per-layer cost is even cheaper than an RNN whenever $n < d$, the common case in sentence-level translation. This is the theoretical backing for claim (b), "training gets faster".

Methodology audit checklist

Audit item	State in this paper	Verdict
Tools/data appropriateness	WMT 2014 EN—DE (~4.5M sentence pairs) and EN—FR (36M); byte-pair / word-piece encodings — the field's standard benchmark.	solid ✓
Samples / repeats (seeds)	Final translation numbers largely from a single training run + checkpoint averaging. No seed variance or confidence intervals reported.	weak
Negative control = baselines	Table 2 compares many of the era's strongest models: ByteNet, ConvS2S, GNMT, MoE.	solid ✓
Positive control = ablations	Table 3 systematically varies heads, d_k , model size, dropout, positional encoding.	solid ✓
Data availability	WMT 2014 is a public, standard dataset.	public ✓
Code availability	Training/evaluation code released as tensorflow/tensor2tensor .	public ✓

STEP 5

Reach independent conclusions

WHAT TO DO AT THIS STEP

Cornwell: "Skip the discussion section; examine results and figures directly to form your own interpretation before reading the author interpretations." Below, the authors' narrative is **deliberately covered up**: we separate what the table's numbers support by themselves from what this data cannot tell us.

Main result — translation quality and training cost

Model	BLEU EN-DE	BLEU EN-FR	Training cost (FLOPs) EN-DE	EN-FR
-------	------------	------------	-----------------------------	-------

ByteNet	23.75	—	—	—
GNMT + RL	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
GNMT + RL <i>Ensemble</i>	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S <i>Ensemble</i>	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base)	27.3	38.1	$3.3 \cdot 10^{18}$	$3.3 \cdot 10^{18}$
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	$2.3 \cdot 10^{19}$

Re-typeset from the paper's Table 2 (WMT 2014 newstest2014); values verified cell by cell. The EN-FR-only Deep-Att+PosUnk rows are omitted for space — single 39.2 / ensemble 40.4.

What the numbers themselves say. Transformer (big)'s EN-DE **28.4** beats every single and ensemble model in the table. Its EN-FR **41.8** is the best single model. Transformer (base) trains for 3.3×10^{18} FLOPs — one to three orders of magnitude below ConvS2S and GNMT (10^{19} — 10^{21}). "Higher BLEU at lower cost" holds from this one table.

What this data alone cannot say. The table says nothing about how well BLEU tracks real translation quality (fluency, meaning preservation). FLOPs are estimates (peak GPU throughput × time) and can differ from wall-clock. Both language pairs are high-resource, so generalization to low-resource languages is unknown.

Questions to ask yourself. ① Is the gain from the architecture, or from the training recipe — bigger batches, label smoothing, checkpoint averaging? ② EN-FR base (38.1) trails some ensembles; in which setting exactly does "SOTA" hold? ③ Is the cost comparison apples-to-apples (hardware and precision assumptions)?

Ablations — what actually moves the needle

Variant	Change (base: $N=6$, $d_{\text{model}}=512$, $h=8$, $P_{\text{drop}}=0.1$)	PPL (dev)	BLEU (dev)
base	—	4.92	25.8
(A)	1 head ($d_k=512$)	5.29	24.9
	32 heads ($d_k=16$)	5.01	25.4
(B)	d_k reduced to 16	5.16	25.1
(C)	bigger model ($d_{\text{model}}=1024$, $d_{\text{ff}}=4096$)	4.66—5.12	25.4—26.2
(D)	dropout 0.0	5.77	24.6
	dropout 0.2	4.95	25.5

(E)	learned positional embedding instead of sinusoids	4.92	25.7
big	$N=6$, $d_{\text{model}}=1024$, $h=16$, $P_{\text{drop}}=0.3$, 300K steps	4.33	26.4

Condensed re-typeset of the paper's Table 3 (EN-DE newstest2013 dev) — representative rows only, values verified cell by cell. See the paper for the full grid.

What the numbers themselves say. (A) One head (24.9) is worse than eight (25.8), and too many (32 $\rightarrow$ 25.4) also hurts — the middle wins. (B) Cutting d_k to 16 drops quality to 25.1 $\rightarrow$ key width matters. (C) Bigger models generally improve. (D) No dropout overfits (24.6); 0.2 is better. (E) Learned positional embeddings (25.7) are effectively tied with sinusoids (25.8).

What this data alone cannot say. These are single runs on a dev set (newstest2013); whether 0.1–0.4 BLEU gaps are statistically meaningful is unknown, as is whether the trends transfer beyond one language pair.

Questions to ask yourself. ① How many repeats would you need before calling a 0.9 BLEU gap "important"? ② If (E) is a tie, the case for sinusoids rests on what, exactly (extrapolation hopes)? ③ Which lever is largest — heads, size, or dropout?

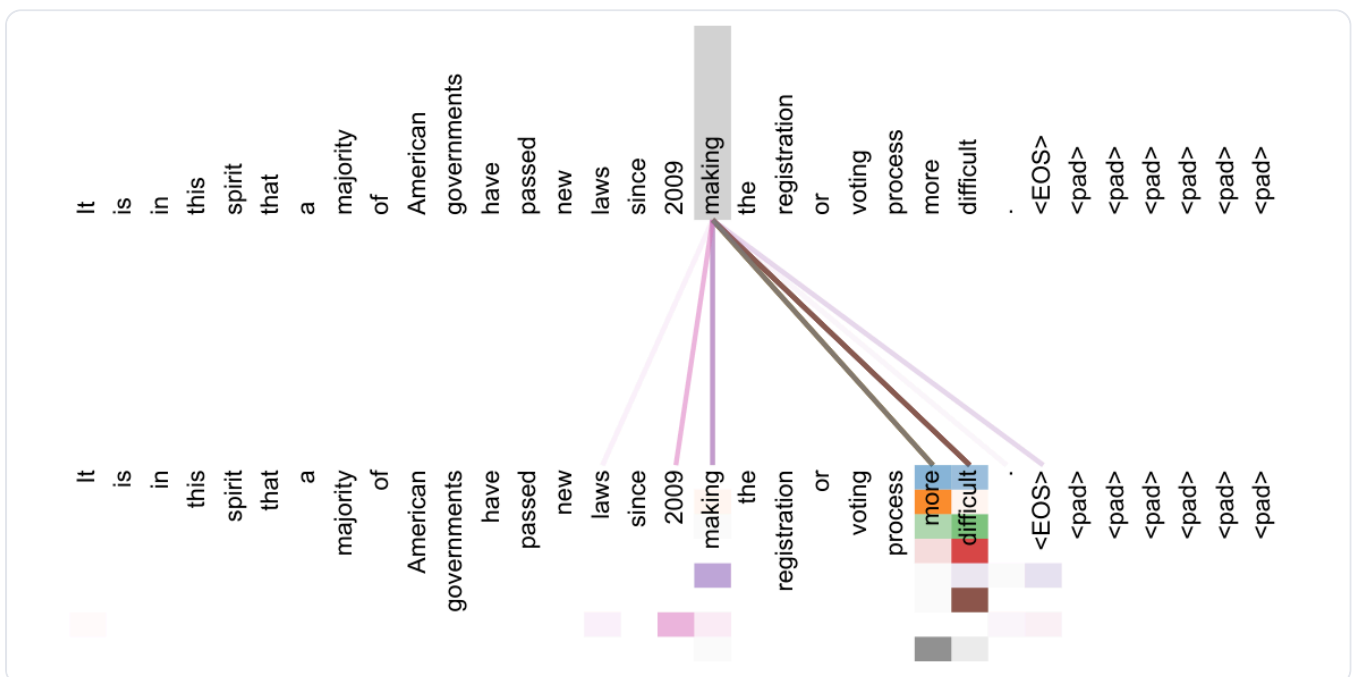


Figure 3 — Encoder self-attention (layer 5 of 6) tracking the long-distance dependency from the verb 'making' to the distant 'more difficult' (colors distinguish heads).

What this figure draws

The attention weights that the word 'making' assigns to other positions in one sentence, drawn as lines between two parallel copies of the sentence, with line darkness showing weight.

Reading order

Follow the lines leaving 'making' in the top row. The dark lines run to the distant words completing the phrase 'making ... more difficult' — a **long-range**, non-adjacent link. Differently colored lines are different heads watching different targets.

In-figure symbols

<EOS> (end of sentence), <pad> (length padding). No mathematical symbols; line opacity = attention weight.

What to observe

Qualitative evidence that a single layer captures a distant grammatical dependency directly, with no recurrence or convolution — Table 1's "path length $O(1)$ " showing up as actual learned long-range structure.

Connections

These are literally the softmax weights of Eq. (1), and they support §4's suggestion that self-attention may yield more interpretable models.

Common sticking points, Top 4

These are the places where first-time readers actually stall. Each is resolved as **the misreading** → **why the paper invites it** → **the resolution**.

Q1. Are Q, K, V three different inputs?

WHY IT TRIPS READERS

Eq. (1) and Figure 2 draw them as three separate input arrows, so they look like three different data sources.

RESOLUTION

In self-attention all three come from **the same input x** — only the projection matrices differ (xW^Q, xW^K, xW^V ; see the MultiHead card). The one place they genuinely come from different sources is encoder—decoder attention: Q from the decoder, K, V from the encoder output (the middle connection in Figure 1).

Q2. "Outputs (shifted right)" — doesn't the decoder peek at the answer?

WHY IT TRIPS READERS

Figure 1's bottom right feeds the target sequence into the decoder, which looks like the ground truth is needed at inference too.

RESOLUTION

During training the target is fed shifted one step (teacher forcing), and masked self-attention (§3.2.3) sets future positions to $-\infty$, so position i 's prediction sees only targets before i . **At inference** the model feeds back its own generated tokens instead. The figure shows the training-time wiring.

Q3. Don't 8 heads cost 8× the compute?

WHY IT TRIPS READERS

"Attention performed 8 times in parallel" (§3.2.2) reads like an $8\times$ bill.

RESOLUTION

Each head is **shrunk** to $d_k = d_v = d_{\text{model}}/h = 64$. Since $8 \times 64 = 512$, the total compute is close to one full-width attention (see the MultiHead card's intuition). What multiplies by 8 is the number of perspectives, not the cost.

Q4. Why is the positional encoding added ($\oplus$) to the embedding rather than concatenated?**WHY IT TRIPS READERS**

Addition feels like it should "smear" word and position information into each other. §3.5 says they are summed but never says why.

RESOLUTION

What the paper states is twofold: PE has the same width d_{model} , so addition is possible (§3.5), and learned embeddings performed the same anyway (Table 3 (E)). Versus concatenation, addition adds no dimensions or parameters. The popular explanation that "in high dimensions the two signals survive in near-orthogonal subspaces" is an interpretation from outside the paper — treat it as **▲ reference only**.

PHASE 3 • THE VERDICT

Final analysis — Steps 6 • 7

STEP 6

Reconciling conclusions — independent reading vs the authors

WHAT TO DO AT THIS STEP

Put the Step-5 independent reading next to the authors' discussion and conclusions. Point out overstated claims and quietly widened scope with **specific sentences and numbers** — and acknowledge agreement where it exists.

Where they agree (fair assessment). Research claims (a) and (b) are well supported. Table 2 shows "higher BLEU at lower FLOPs" and Table 1 shows " $O(1)$ sequential ops and path length", so the core claim — SOTA translation and fast training from attention alone, no recurrence — is not an overstatement. The ablations (Table 3) also control each component's contribution honestly.

Points to press.

- **The title oversells.** "Attention Is All You Need" — yet the model does not work without the position-wise FFN (Eq. 2), residuals, LayerNorm, and positional encoding. Table 3 (D)'s drop at dropout 0.0 (24.6) shows non-attention pieces are also essential. Read the title as rhetoric, not as a literal claim.
- **An internal inconsistency.** §6.1 of the paper reports the EN-FR big model as "a BLEU score of 41.0", while Table 2 lists **41.8**. Which number is final is ambiguous from the text alone (the abstract and table use 41.8).
- **No statistical significance.** Table 3's 0.1–0.4 BLEU gaps carry conclusions like "bigger is better" and " d_k matters", but with no repeats or variance reported, some of these gaps may be noise.

STEP 7

Considering alternative explanations

WHAT TO DO AT THIS STEP

Check confounders that could explain the results, limitations the authors do and do not acknowledge, and conflicts of interest / funding.

Category	Assessment
----------	------------

Confounders	How much of the gain is the self-attention architecture versus the training recipe — large batches ($\approx 25k$ tokens), label smoothing ($\epsilon_{ls}=0.1$) ^[K10] , checkpoint averaging, the warm-up schedule — is not fully separable. Table 3 controls architectural knobs, but the recipe stays mostly fixed.
Metric dependence	Every conclusion rides on BLEU alone; whether human evaluation or other metrics would preserve the ranking is unverified.
Limitations acknowledged	$O(n^2)$ attention cost on long sequences → the authors themselves preview "restricted (local) attention"; extending beyond text (images, audio, video) and making generation less sequential are left as future work.
Limitations not acknowledged	No seed variance or significance reporting; validation limited to high-resource language pairs; no quantification of what $O(n^2)$ memory means for real deployments.
Conflicts / funding	Seven of eight authors are at Google (Brain/Research) and the released code is Google's <code>tensor2tensor</code> . No separate funding or COI statement. Benchmarks and cost estimates are on in-house infrastructure — worth keeping in mind while reading.

Final insight & self-check

The core contribution, reconstructed in four sentences. The Transformer is the first sequence-transduction model to remove recurrence and convolution entirely, building its encoder—decoder from self-attention that links any two positions in a constant number of operations. It parallelizes scaled dot-product attention across multiple heads, injects order through sinusoidal positional encodings, reprocesses each position with a feed-forward network, and stabilizes everything with residuals and LayerNorm. The result trained massively in parallel and set a new SOTA on WMT 2014 EN-DE (28.4) and EN-FR (41.8) at a fraction of prior cost, generalizing to parsing as well. This simple, parallel architecture went on to become the backbone of virtually every large language and vision model.

Seven-question self-check — can you answer each in your own words?

- ☐ (S1) Can you explain the encoder—decoder data flow from the one architecture figure?
- ☐ (S2) Can you state the paper's hypothesis — (a) quality, (b) speed — in one sentence?
- ☐ (S3) Can you distinguish the RNN/CNN lineages' limits from the gap the Transformer fills?
- ☐ (S4) Can you give the role of each term in Eq. (1) — $QK^T/\sqrt{d_k}$, softmax, V — and the reason for $\sqrt{d_k}$?
- ☐ (S5) Can you draw conclusions from Tables 2—3 without the authors' narration?
- ☐ (S6) Can you spot the title's overreach and the 41.0 / 41.8 inconsistency yourself?
- ☐ (S7) Can you name the alternative explanations (training recipe, metric dependence) and the conflicts of interest?

Appendix — Run it yourself

Reproduction code for the numeric examples in this report (requires numpy). Copy, run, and you should see the same output. ✓ All four snippets passed machine verification when this report was generated.

① The [0.73, 0.27] from Eq. (1)'s intuition — Scaled Dot-Product Attention card

```
import numpy as np
scores = np.array([8.0, 0.0])          # query · key dot-product scores
scaled = scores / np.sqrt(64)          # d_k = 64 → divide by 8
w = np.exp(scaled) / np.exp(scaled).sum() # softmax
print(np.round(w, 2))
```

```
[0.73 0.27]
```

② Ladder rung 3's soft max — [2,1,0] → weights

```
import numpy as np
z = np.array([2.0, 1.0, 0.0])
w = np.exp(z) / np.exp(z).sum()
print(np.round(w, 3), round(w.sum(), 3)) # weights and their sum (=1)
```

```
[0.665 0.245 0.09 ] 1.0
```

③ Ladder rung 10 / Eq. (1)'s $\sqrt{d_k}$ argument — is the dot product's std really 8?

```
import numpy as np
rng = np.random.default_rng(0)          # fixed seed → reproducible
q, k = rng.standard_normal((2, 100000, 64))
dots = (q * k).sum(axis=1)              # 100k dot products at d_k = 64
print(round(dots.std(), 1), round((dots / np.sqrt(64)).std(), 2))
```

```
8.0 1.0
```

④ Ladder rung 8's LayerNorm — [2,4,6] → normalized

```
import numpy as np
x = np.array([2.0, 4.0, 6.0])
ln = (x - x.mean()) / x.std()
print(np.round(ln, 2))
```

```
[-1.22  0.    1.22]
```

Glossary

Term	Meaning
------	---------

BLEU	Automatic machine-translation metric based on n-gram overlap with reference translations (higher is better).
BPE / word-piece	Subword tokenizers that split words into frequent pieces to handle rare and unseen words.
encoder—decoder attention	Attention where the decoder provides queries and the encoder output provides keys/values; generation consults the input.
FFN (position-wise)	A two-layer fully connected network applied identically at each position (Eq. 2); no cross-position interaction.
LayerNorm	Normalizes a sample's features to zero mean and unit variance, stabilizing training (the "Norm" in Add & Norm).
masked self-attention	Decoder self-attention with future positions set to $-\infty$, preserving autoregressive generation.
multi-head attention	Running attention h times through different projections and merging the results (the MultiHead formula).
positional encoding	Sinusoidal vectors injecting order information into a recurrence-free model (§3.5).
residual connection	Adding the input to the output, $\mathbf{x} + \text{Sublayer}(\mathbf{x})$, to ease training of deep networks.
scaled dot-product attention	$\text{softmax}(QK^T/\sqrt{d_k})V$ — the Transformer's basic attention unit (Eq. 1).
self-attention	Attention relating different positions of one sequence to build that sequence's representation.
sequence transduction	Transforming one sequence into another (translation, summarization, ...).
warmup	Linearly raising the learning rate early in training before decaying it (Eq. 3).

References

The paper. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. "Attention Is All You Need." NIPS 2017. arXiv:1706.03762. <https://arxiv.org/abs/1706.03762>

The [K#] entries below are external sources used for background and related work. All were verified directly against their arXiv abstract or official page (checked 2026-07-06; [K3]'s publisher abstract returned 403, so only its bibliography was confirmed). ✓ **Verified**

ID	Source (title • authors • year • URL)	Status
[K1]	Neural Machine Translation by Jointly Learning to Align and Translate — Bahdanau, Cho, Bengio, 2014. arXiv:1409.0473. link	✓
[K2]	Sequence to Sequence Learning with Neural Networks — Sutskever, Vinyals, Le, 2014 (NIPS). arXiv:1409.3215. link	✓
[K3]	Long Short-Term Memory — Hochreiter, Schmidhuber, 1997. Neural Computation 9(8):1735—1780. link	✓ biblio only
[K4]	Convolutional Sequence to Sequence Learning — Gehring, Auli, Grangier, Yarats, Dauphin, 2017. arXiv:1705.03122. link	✓
[K5]	Layer Normalization — Ba, Kiros, Hinton, 2016. arXiv:1607.06450. link	✓
[K6]	Neural Machine Translation in Linear Time (ByteNet) — Kalchbrenner et al., 2016. arXiv:1610.10099. link	✓
[K7]	Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer — Shazeer et al., 2017. arXiv:1701.06538. link	✓
[K8]	Adam: A Method for Stochastic Optimization — Kingma, Ba, 2014 (ICLR 2015). arXiv:1412.6980. link	✓
[K9]	Google's Neural Machine Translation System (GNMT) — Wu, Schuster, Chen et al., 2016. arXiv:1609.08144. link	✓
[K10]	Rethinking the Inception Architecture for Computer Vision (label smoothing) — Szegedy et al., 2015 (CVPR 2016). arXiv:1512.00567. link	✓
[K11]	Deep Residual Learning for Image Recognition — He, Zhang, Ren, Sun, 2015 (CVPR 2016). arXiv:1512.03385. link	✓